AI Agent-Based Autonomous Ticket Booking Application

Team: The Xplorers | B.E. Computer Science and Engineering (First Year) | Team Size: 4

1. Project Overview

A conversational website where users describe travel needs in plain language and an AI agent helps search, compare, confirm, and book tickets. The initial version focuses on **flights**; other ticket types may be added later.

2. Aim

Build a working flight-booking website with AI chat, real booking-API integration in **test/sandbox mode**, user confirmation, booking status/history, and cancellation where supported. A public live-booking version is a future goal.

3. Problem Statement

Travel booking requires users to compare fares and schedules, enter passenger information, and complete multiple steps. Real bookings, payments, and cancellations are more complex than simply showing flight results.

4. Proposed Solution / Workflow

User enters trip details → AI extracts origin, destination, date, passenger count and class → asks for missing details → searches flight options through the API → shows fares/timings → collects passenger details and **explicit confirmation** → creates a sandbox order → displays booking reference/status and history.

5. Core Concepts

AI agents and function/tool calling; slot filling and input validation; REST API and client–server architecture; authentication; database storage with row-level security; responsive web design.

6. Technologies Used

Next.js (frontend and backend API routes); **Tailwind CSS + shadcn/ui** (interface); **Vercel AI SDK** (chat and tools); **Supabase/PostgreSQL** (database and login); **Duffel Flights API** (sandbox flight search/booking); **Vercel** (hosting). Later: PWA/Capacitor for mobile.

7. Programming Languages & Essential Tools

TypeScript (main application and AI logic), **JavaScript** (ecosystem), **SQL** (database), **HTML/CSS** (web structure and styling). **JSON** is an API data format, not a programming language. Tools: **Node.js, npm, Git/GitHub, VS Code** and an API tester (e.g., Postman). **Python is optional, not required.**

8. Essential Safety & Testing

Keep API keys on the server; validate inputs and check fare/availability again before booking; require user approval; prevent duplicate orders; handle API failures; protect passenger information. Clearly mark sandbox tickets **TEST — NOT VALID FOR TRAVEL**. Real payments/bookings require production API approval and additional security checks.